

Offline-First POS Platform — Final Architecture & AI Development Blueprint

Vision

Build a Vyapar-like modern commerce platform that supports:

- Desktop POS application
- Mobile application
- Cloud backend
- Offline-first operation
- Multi-counter retail stores
- Real-time local synchronization
- Cloud synchronization
- Accounting + inventory support
- SaaS multi-tenant architecture

The platform must continue operating even if internet connectivity is unavailable for extended periods.

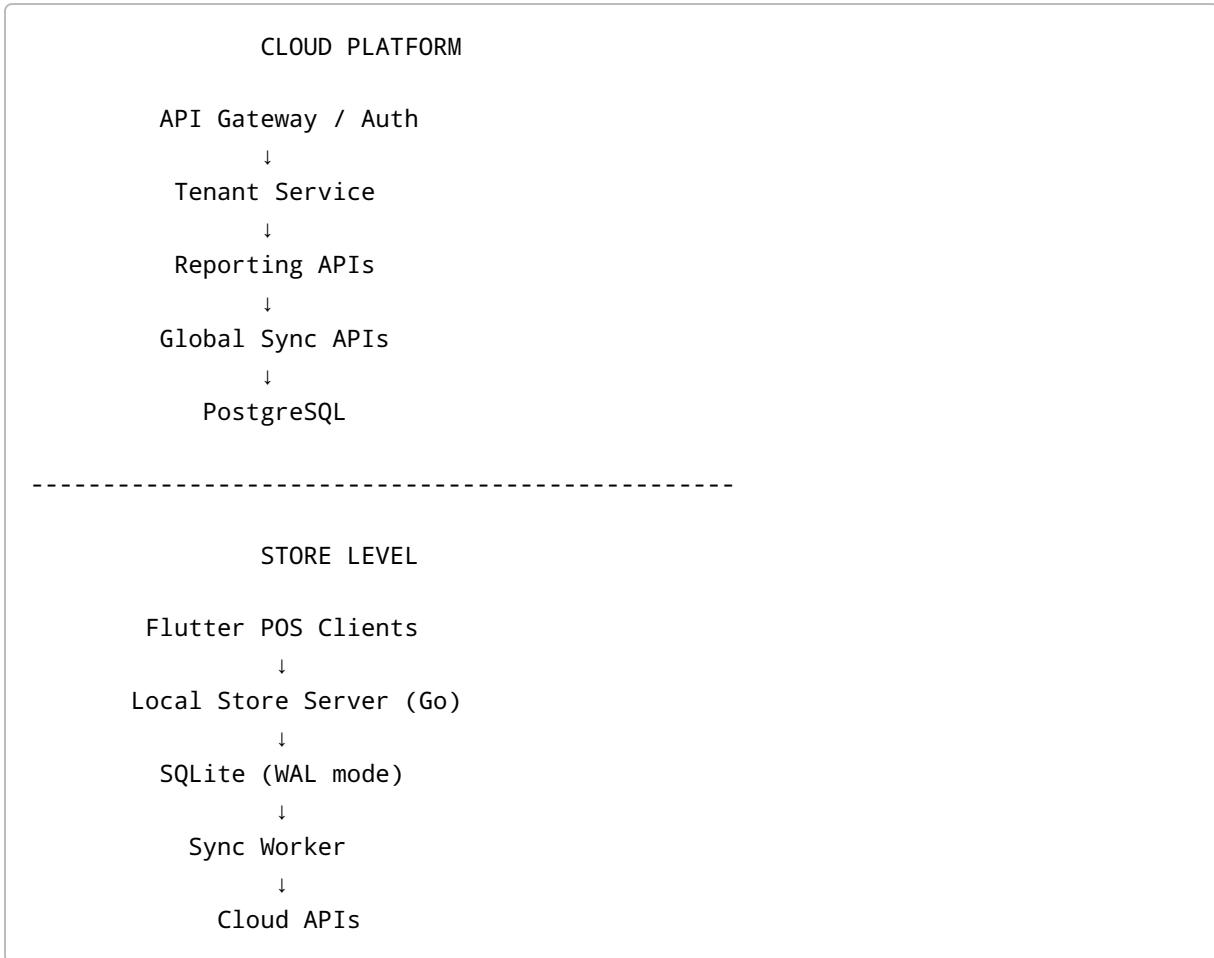
1. Core Architectural Philosophy

Golden Rules

1. Local store always works
 2. Cloud is asynchronous
 3. Local operations are source of truth
 4. Internet is optional
 5. Sync must be idempotent
 6. Inventory changes are append-only ledger events
 7. Finalized invoices must never be edited
 8. Multi-counter consistency is owned by local store server
 9. Sync events are more important than current state snapshots
 10. Event log is the heart of the system
-

2. Final Recommended Architecture

High-Level Architecture



3. Technology Stack

Frontend

Flutter

Used for:

- Windows desktop POS
- macOS desktop POS
- Mobile application

Why Flutter

- Single codebase
 - Native performance
 - Excellent SQLite support
 - Good printer support
 - Barcode scanner support
 - Works offline naturally
 - Strong future maintainability
-

Backend

Go (Golang)

Used for:

- Local store server
- Cloud backend APIs
- Sync workers
- WebSocket communication
- Event processing

Why Go

- Lightweight binaries
 - Easy deployment
 - Excellent concurrency
 - Reliable long-running services
 - Very low memory usage
 - Good SQLite/PostgreSQL ecosystem
 - Perfect for event-driven systems
-

Databases

Local Store

SQLite (WAL Mode)

Why:

- Zero configuration
- Reliable
- Fast
- Easy backup

- Works perfectly for SME scale
- Single-file DB

Mandatory:

```
PRAGMA journal_mode=WAL;
```

Cloud Database

PostgreSQL

Why:

- Strong transactional guarantees
 - Financial consistency
 - Reporting support
 - Reliable indexing
 - Multi-tenant friendly
 - Mature ecosystem
-

4. Deployment Model

Every Store Has Local Store Server

Even single-counter stores.

Why

This standardizes architecture.

Benefits:

- One sync model
- One deployment model
- One inventory ownership model
- Easier debugging
- Easier maintenance
- Easier scaling

Single-counter stores become a simplified subset of the same architecture.

5. Multi-Tenant Architecture

Recommendation: Separate Database Per Tenant

Why

- Better isolation
- Easier backup
- Easier migration
- Better security
- Easier scaling
- Enterprise friendly

Structure

```
Tenant A → DB A  
Tenant B → DB B  
Tenant C → DB C
```

6. Repository & Project Structure

Monorepo Structure

```
repo/  
  
  apps/  
    pos_flutter/  
    mobile_flutter/  
    local_store_server/  
    cloud_api/  
    admin_dashboard/  
  
  packages/  
    shared_models/  
    event_contracts/  
    sync_sdk/  
    auth_sdk/  
    inventory_sdk/  
    accounting_sdk/  
  
  docs/  
    architecture/
```

```
sync_rules/  
inventory_rules/  
accounting_rules/  
api_contracts/  
event_contracts/  
deployment/  
  
infra/  
  docker/  
  kubernetes/  
  terraform/  
  
scripts/  
  local_dev/  
  migrations/  
  backup/
```

7. Core Services

Local Store Server

Responsibilities:

- Local inventory ownership
- Local transaction processing
- WebSocket real-time updates
- Receipt generation
- Local sync queue
- Offline operations
- Counter synchronization

Cloud API Service

Responsibilities:

- Authentication
 - Tenant management
 - Reporting
 - Cross-branch aggregation
 - Mobile dashboards
 - Global sync ingestion
-

Sync Service

Responsibilities:

- Retry operations
 - Idempotency checks
 - Conflict handling
 - Sync batching
 - Operation ordering
-

8. Event-Driven Design

Important Principle

Never sync final state.

BAD:

```
stock = 5
```

GOOD:

```
item_received +5  
item_sold -1  
refund +1
```

Example Events

```
sale_created  
payment_added  
payment_refunded  
inventory_adjusted  
stock_transferred  
sync_completed  
sync_failed  
user_logged_in
```

9. Core Database Tables

operations_log

Purpose:

- Sync source
- Retry queue
- Audit trail

Fields:

```
id
operation_id (UUID)
operation_type
entity_type
entity_id
payload
created_at
sync_status
origin_node_id
retry_count
```

inventory_movements

Append-only inventory ledger.

Never update inventory directly.

payments

Separate from invoices.

Supports:

- split payments
 - partial payments
 - refunds
-

sync_state

Tracks:

- last sync
 - sync checkpoints
 - failed operations
-

10. Real-Time Multi-Counter Sync

Use WebSocket

Flow:

```
POS1 sells item
  ↓
Local Store Server
  ↓
WebSocket broadcast
  ↓
POS2 inventory updates instantly
```

11. Offline Strategy

Local Store Always Wins

Why

POS must never block sales.

Rules:

- local sales are accepted immediately
 - cloud never rejects completed local sales
 - cloud recalculates global state later
-

12. Sync Strategy

Idempotency is Mandatory

Every operation MUST have:

```
operation_uuid
```

Cloud checks:

```
if operation_exists:  
    ignore
```

Sync Batching

Sync related operations atomically.

Example:

- invoice
- payment
- stock movement

Must succeed together.

13. Edge Cases You MUST Handle

Sync Edge Cases

- duplicate sync
 - out-of-order sync
 - partial sync failure
 - reconnect storms
 - clock drift
 - replay attacks
-

Inventory Edge Cases

- two counters sell last item simultaneously
 - negative stock
 - stale cache
 - stock transfer conflicts
-

Payment Edge Cases

- payment success but invoice failure
 - split payments
 - refund after sync
 - duplicate payment retry
-

Device Failures

- local server crash
 - corrupted DB
 - power failure during write
 - POS app crash mid-sale
-

Multi-Tenant Risks

- tenant data leakage
 - shared cache pollution
 - auth token misuse
-

14. AI-Assisted Development Rules

Most Important Rule

Architecture decisions are human-owned.

AI should:

- implement
- refactor
- generate tests
- generate docs
- generate UI

AI should NOT fully own:

- sync logic
 - accounting correctness
 - inventory consistency
 - conflict rules
-

15. Mandatory Documentation

Create these docs EARLY.

```
/docs
  sync-rules.md
  inventory-rules.md
  accounting-rules.md
  event-contracts.md
  tenant-rules.md
  security-rules.md
```

16. Invariants (Critical)

These rules must NEVER be violated.

- finalized invoice cannot change
 - inventory ledger is append-only
 - sync operations are idempotent
 - refunds are reversal events
 - local completed sales are never rejected
-

17. API & Model Strategy

Recommendation

Use:

- OpenAPI OR

- Protobuf

Generate:

- Go models
- Flutter models
- API clients

Never manually duplicate models.

18. Security Considerations

Required

- JWT authentication
 - refresh token rotation
 - device registration
 - tenant isolation
 - encrypted sync transport
 - signed operations
-

19. Logging & Observability

Every important event must be logged.

Example:

```
sale_created  
payment_added  
sync_started  
sync_failed  
inventory_locked  
refund_completed
```

20. Testing Strategy

Unit Tests

For:

- tax calculations
 - inventory logic
 - accounting logic
 - sync ordering
-

Integration Tests

For:

- sync retry
 - reconnect
 - multi-counter inventory
-

Chaos Testing

Simulate:

- internet disconnect
 - duplicate sync
 - power failure
 - server crash
 - stale cache
-

21. Build Order (VERY IMPORTANT)

Phase 1 — Core Engine

Build FIRST:

1. operation log
2. sync engine
3. inventory ledger
4. local DB architecture
5. event contracts

DO NOT start with UI.

Phase 2 — Basic POS

Build:

- sales screen
 - inventory
 - receipt printing
 - barcode scanning
 - local persistence
-

Phase 3 — Sync

Build:

- cloud sync
 - retry logic
 - idempotency
 - reconnect handling
-

Phase 4 — Multi-Counter

Build:

- WebSocket updates
 - inventory locks
 - live cart updates
-

Phase 5 — Accounting & Reports

Build:

- ledger
 - tax reports
 - owner dashboard
-

22. Recommended Development Workflow with AI

Good Prompt Example

Implement sync retry worker.

Requirements:

- idempotent operations
- exponential backoff
- append-only operation log
- local store always wins
- max retry count 10
- operation batching

Bad Prompt Example

Build POS sync system

23. Recommended Initial MVP

MUST HAVE

- local sales
- local inventory
- receipt printing
- sync queue
- cloud sync
- multi-counter updates

DO NOT BUILD EARLY

- HR
- payroll
- CRM
- loyalty
- advanced analytics
- marketplace

24. Recommended Infrastructure

Local Store Server

Deployment:

```
Single Go binary  
+ SQLite file
```

Benefits:

- easy install
 - easy backup
 - low memory
 - low maintenance
-

Cloud

Initial

- VPS
- Docker
- PostgreSQL

Later

- Kubernetes
 - autoscaling
 - managed PostgreSQL
-

25. Biggest Long-Term Risks

Technical Risks

- sync inconsistency
 - inventory corruption
 - duplicate operations
 - stale cache bugs
 - event ordering bugs
-

Product Risks

- overbuilding too early
- ERP complexity
- poor sync UX
- weak observability

26. Final Strategic Advice

This is NOT a normal CRUD app.

This is:

Distributed retail synchronization platform

Your competitive advantage will NOT be:

- UI
- reports
- dashboards

It will be:

Reliable offline-first synchronization

Focus engineering effort there.

27. Final Recommended Stack Summary

Layer	Technology
Desktop App	Flutter
Mobile App	Flutter
Local Store Server	Go
Cloud Backend	Go
Local DB	SQLite
Cloud DB	PostgreSQL

Layer	Technology
Realtime	WebSocket
Contracts	OpenAPI / Protobuf
Sync	Event-driven operation log
Architecture	Monorepo
Hosting	SaaS multi-tenant

28. Final Recommendation

The best initial path is:

1. Build local store server first
2. Build operation log and sync engine
3. Build minimal POS UI
4. Add cloud synchronization
5. Add multi-counter live updates
6. Add accounting and reporting later

Do not overbuild initially.

Prioritize:

- sync correctness
- inventory consistency
- reliability
- observability

Everything else can evolve later.